

Asztali és mobil alkalmazások fejlesztése és tesztelése tantárgy – Mobil alkalmazások rész

13. A

Tanárok: Dr. Koncz Adrienn
és Simon Martin

2026.02.26.





React Navigation

Mi az a React Navigation?

- A React Navigation egy nyílt forráskódú könyvtár, amelyet React Native alkalmazásokban használnak a képernyők közötti navigáció megvalósítására.
- Egy mobilalkalmazás tipikusan több screenből épül fel, ezért szükség van egy olyan megoldásra, amely kezeli, hogyan lehet az egyes nézetek között váltani, miként szervezhetők a képernyők különböző navigációs struktúrákba (például stack, tab vagy drawer), valamint hogyan adhatók át adatok a navigáció során.
- A **React Navigation** mindehhez könnyen használható eszközöket ad: előre elkészített komponenseket és egyszerűen érthető megoldásokat (API-k) biztosít, amelyek jól működnek a React Native alkalmazásokban.
- Ennek köszönhetően a fejlesztők viszonylag kevés kóddal is átlátható és jól működő navigációt tudnak kialakítani az appjukban.

Kötelező importok

```
import { createStaticNavigation } from  
'@react-navigation/native';
```

```
import { createNativeStackNavigator } from  
'@react-navigation/native-stack';
```

<https://reactnavigation.org/docs/hello-react-navigation?config=static>



Az importokban szereplő függvények

- A **createNativeStackNavigator** és a **createStaticNavigation** itt most nem komponensek, hanem **függvények**.
- A **createNativeStackNavigator** egy konfigurációs objektumot vár paraméterként, amely megadja, hogy mely screenek tartozzanak a navigatorhoz, valamint további beállítási lehetőségeket is tartalmazhat.
- A **createStaticNavigation** a navigatorból egy olyan komponenst hoz létre, amelyet az alkalmazásban lehet renderelni. Ezt a függvényt általában csak egyszer hívjuk meg, jellemzően az app gyökerében (root).

Hozzuk létre az alap navigator kód szerekezetet

```
import * as React from 'react';
import { View, Text } from 'react-native';
import { createStaticNavigation } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

function HomeScreen() {
  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>Home Screen</Text>
    </View>
  );
}

const RootStack = createNativeStackNavigator({
  screens: {
    Home: HomeScreen,
  },
});

const Navigation = createStaticNavigation(RootStack);

export default function App() {
  return <Navigation />;
}
```

A navigator konfigurálása

- Eddig még nem adtunk át semmilyen beállítást a navigatornak, ezért jelenleg az alapértelmezett (default) konfigurációt használja.
- Most adjunk hozzá egy második screent, és állítsuk be a Home screent kezdő route-ként (initial route).

```
function DetailsScreen() {  
  return (  
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>  
      <Text>Details Screen</Text>  
    </View>  
  );  
}  
  
const RootStack = createNativeStackNavigator({  
  initialRouteName: 'Home',  
  screens: {  
    Home: HomeScreen,  
    Details: DetailsScreen,  
  },  
});
```

Beállítások megadása

- Minden screenhez megadhatunk különféle beállításokat (options), például a header címét.
- A screen konfigurációjában az **options** tulajdonság segítségével állíthatjuk be az adott screenre vonatkozó beállításokat.

```
const RootStack = createNativeStackNavigator({  
  initialRouteName: 'Home',  
  screens: {  
    Home: {  
      screen: HomeScreen,  
    },  
    Details: DetailsScreen,  
  },  
});
```

Beállítások megadása

- Most már hozzáadhatunk egy options tulajdonságot.

```
const RootStack = createNativeStackNavigator({  
  initialRouteName: 'Home',  
  screens: {  
    Home: {  
      screen: HomeScreen,  
      options: {  
        title: 'Overview',  
      },  
    },  
    Details: DetailsScreen,  
  },  
});
```



Beállítások megadása

Fontos!

- További propok átadása egy screen számára a statikus API használata esetén nem támogatott.

- Ha ugyanazokat a beállításokat szeretnénk alkalmazni minden screenre, a navigatoron a **screenOptions** tulajdonságot használhatjuk.

```
const RootStack = createNativeStackNavigator({
  initialRouteName: 'Home',
  screenOptions: {
    headerStyle: { backgroundColor: 'tomato' },
  },
  screens: {
    Home: {
      screen: HomeScreen,
      options: {
        title: 'Overview',
      },
    },
    Details: DetailsScreen,
  },
});
```



Navigáció a képernyők között

- A React Navigation használatakor két alapvető módon válthatunk a screenek között:
 - **Link** vagy **Button** komponensek használata
- A navigáció legegyszerűbb módja:
 - a Link komponens használata az **@react-navigation/native** könyvtárból,
 - a Button komponens az **@react-navigation/elements** könyvtárból.
- A Link és a Button komponensek rendelkeznek egy screen tulajdonsággal, amely megadja, hogy melyik screenre történjen a navigáció a megnyomáskor.
- A beépített Link és Button komponensek saját stílusokkal rendelkeznek.

Navigáció a képernyők között

```
import * as React from 'react';
import { View, Text } from 'react-native';
import { createStaticNavigation } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { Link } from '@react-navigation/native';
import { Button } from '@react-navigation/elements';

function HomeScreen() {
  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>Home Screen</Text>
      <Link screen="Details">Go to Details</Link>
      <Button screen="Details">Go to Details</Button>
    </View>
  );
}
```

A navigation objektum használata

- A képernyők közötti váltás másik gyakori módja a navigation objektum használata.
- Ez a megoldás több lehetőséget biztosít, és pontosabban szabályozhatjuk vele, hogy mikor és hogyan történjen a navigáció.
- A navigation objektum a screen komponenseken belül a **useNavigation** hook segítségével érhető el.



A navigation objektum használata

```
import * as React from 'react';
import { View, Text } from 'react-native';
import {
  createStaticNavigation,
  useNavigation,
} from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { Button } from '@react-navigation/elements';

function HomeScreen() {
  const navigation = useNavigation();

  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>Home Screen</Text>
      <Button onPress={() => navigation.navigate('Details')}>
        Go to Details
      </Button>
    </View>
  );
}
```

A useNavigation hook a navigation objektumot adja vissza.
A navigate metódus meghívásával, a kívánt route nevének megadásával tudunk másik screenre navigálni.

Navigálás többször ugyanarra a screenre

Mi történik akkor, ha a Details screenre navigálunk, miközben már eleve a Details screenen vagyunk?

```
function DetailsScreen() {  
  const navigation = useNavigation();  
  
  return (  
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>  
      <Text>Details Screen</Text>  
      <Button onPress={() => navigation.navigate('Details')}>  
        Go to Details... again  
      </Button>  
    </View>  
  );  
}
```

Navigálás többször ugyanarra a screenre

- A „Go to Details... again” gomb megnyomásakor nem történik semmi, mert már ezen a route-on tartózkodunk.
- A navigate függvény jelentése lényegében az, hogy „menj erre a screenre” – ha már ott vagyunk, nem vált át újra.
- Tegyük fel viszont, hogy valóban szeretnénk egy újabb Details screent megnyitni.
- Ez gyakori helyzet, például amikor minden route-hoz különböző adatokat adunk át.
- Ilyenkor a navigate helyett a **push metódust** érdemes használni.
- A push minden esetben hozzáad egy új route-ot, függetlenül a korábbi navigációs előzményektől.

Navigálás többször ugyanarra a screenre

```
<Button onPress={() => navigation.push('Details')}>  
  Go to Details... again  
</Button>
```

- Minden push hívás egy új route-ot helyez a stack tetejére.
 - A navigate ezzel szemben csak akkor ad hozzá új route-ot, ha az adott route még nem aktív (vagyis ha éppen nem azon a screenen vagyunk).
-

Visszagomb

- A native stack navigator fejlécében (header) automatikusan megjelenik egy vissza gomb, ha van olyan képernyő, ahová vissza lehet lépni.
- A **navigation.goBack()** metódussal saját gombból is elindíthatjuk a visszalépést programozott módon.

```
function DetailsScreen() {  
  const navigation = useNavigation();  
  
  return (  
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>  
      <Text>Details Screen</Text>  
      <Button onPress={() => navigation.push('Details')}>  
        Go to Details... again  
      </Button>  
      <Button onPress={() => navigation.goBack()}>Go back</Button>  
    </View>  
  );  
}
```

Visszagomb

- Előfordulhat, hogy egyszerre több screenre szeretnénk visszalépni.
- Például ha egy stackben több képernyőn keresztül navigáltunk, és közvetlenül az első screenre szeretnénk visszatérni.
- Erre két lehetőségünk van:
 - **navigation.popTo('Home')** – Visszalépés egy megadott screenre (jelen esetben a Home screenre)
 - **navigation.popToTop()** – Visszalépés a stack legelső screenjére

```
function DetailsScreen() {  
  const navigation = useNavigation();  
  
  return (  
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>  
      <Text>Details Screen</Text>  
      <Button onPress={() => navigation.push('Details')}>  
        Go to Details... again  
      </Button>  
      <Button onPress={() => navigation.goBack()}>Go back</Button>  
      <Button onPress={() => navigation.popTo('Home')}>Go to Home</Button>  
      <Button onPress={() => navigation.popToTop()}>  
        Go back to first screen in stack  
      </Button>  
    </View>  
  );  
}
```



Paraméterek átadása a route-oknak

A paraméterek átadása a route-oknak kettő részből áll :

1. A paramétereket a navigation metódusok második argumentumaként adhatjuk meg:
navigation.navigate('RouteName', { /* ide kerülnek a paraméterek */ })
2. A paramétereket a screen komponensen belül a **route.params** segítségével olvashatjuk ki.

```
function HomeScreen() {
  const navigation = useNavigation();

  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>Home Screen</Text>
      <Button
        onPress={() => {
          /* 1. Navigate to the Details route with params */
          navigation.navigate('Details', {
            itemId: 86,
            otherParam: 'anything you want here',
          });
        }}
      >
        Go to Details
      </Button>
    </View>
  );
}
```

```
function DetailsScreen({ route }) {
  const navigation = useNavigation();

  /* 2. Get the param */
  const { itemId, otherParam } = route.params;

  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>Details Screen</Text>
      <Text>itemId: {JSON.stringify(itemId)}</Text>
      <Text>otherParam: {JSON.stringify(otherParam)}</Text>
      <Button
        onPress={() => {
          navigation.push('Details', {
            // Randomly generate an ID for demonstration purposes
            itemId: Math.floor(Math.random() * 100),
          })
        }}
      >
        Go to Details... again
      </Button>
      <Button onPress={() => navigation.navigate('Home')}>Go to Home</Button>
      <Button onPress={() => navigation.goBack()}>Go back</Button>
    </View>
  );
}
```

Kezdeti paraméterek

```
{  
  Details: {  
    screen: DetailsScreen,  
    initialParams: { itemId: 42 },  
  },  
}
```

- Kezdeti paramétereket az **initialParams** beállítással adhatunk meg.
- Ezek akkor kerülnek felhasználásra, amikor egy screenre paraméterek nélkül navigálunk, és automatikusan kiegészülnek a később átadott paraméterekkel.
- Ha navigáció közben is adunk meg paramétereket, azok felülírják vagy kibővítik a kezdeti értékeket

Paraméterek frissítése

```
navigation.setParams({  
  itemId: Math.floor(Math.random() * 100),  
})
```

+

•

○

- A screenek a paramétereiket a **navigation.setParams** metódussal módosíthatják.
- A setParams az új paramétereket összevonja a meglévőkkel.
- Ha a paramétereket teljes egészében le szeretnénk cserélni, a **replaceParams** metódust használjuk.
- Megjegyzés: A **setParams** és a **replaceParams** használata nem ajánlott a screen beállításainak (például a címnek) módosítására.
- Ilyen esetekben a **setOptions** metódust érdemes használni.

Paraméterek átadása egy korábbi screennek

- Paramétereket nemcsak előre navigáláskor, hanem egy korábbi screen számára is átadhatunk.
- Ez például akkor hasznos, amikor egy „Create post” gomb megnyit egy új képernyőt, és az ott létrehozott adatokat vissza szeretnénk küldeni.
- Ilyenkor a **popTo** metódust használhatjuk: ezzel visszalépünk a kívánt screenre, és közben paramétereket is átadhatunk neki.

```
function HomeScreen({ route }) {  
  const navigation = useNavigation();  
  
  // Use an effect to monitor the update to params  
  React.useEffect(() => {  
    if (route.params?.post) {  
      // Post updated, do something with `route.params.post`  
      // For example, send the post to the server  
      alert('New post: ' + route.params?.post);  
    }  
  }, [route.params?.post]);  
  
  return (  
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>  
      <Button onPress={() => navigation.navigate('CreatePost')}>  
        Create post  
      </Button>  
      <Text style={{ margin: 10 }}>Post: {route.params?.post}</Text>  
    </View>  
  );  
}
```

Paraméterek átadása egy korábbi screennek

A „Done” gomb megnyomása után a Home screen **route.params** értéke frissülni fog a bejegyzés szövegével.

```
function CreatePostScreen({ route }) {  
  const navigation = useNavigation();  
  const [postText, setPostText] = React.useState('');  
  
  return (  
    <>  
      <TextInput  
        multiline  
        placeholder="What's on your mind?"  
        style={{ height: 200, padding: 10, backgroundColor: 'white' }}  
        value={postText}  
        onChangeText={setPostText}  
      />  
      <Button  
        onPress={() => {  
          // Pass params back to home screen  
          navigation.popTo('Home', { post: postText });  
        }}  
      >  
        Done  
      </Button>  
    </>  
  );  
}
```



Kitérő rész:

Mi az a useEffect?

- useEffect hook a React Native egyik legfontosabb és leggyakrabban használt hookja.
- Segítségével úgynevezett side effecteket kezelhetünk funkcionális komponensekben, például adatlekérést végezhetünk, eseményfeliratkozásokat hozhatunk létre, vagy különböző műveleteket hajthatunk végre a renderelés után.
- Egyszerűen megfogalmazva, a useEffect lehetővé teszi, hogy bizonyos kódok a komponens kirajzolása (renderelése) után fussanak le.
- Legyen szó API-hívásról vagy eseménykezelésről, a useEffect az egyik leggyakoribb eszköz az ilyen feladatokra.

- Az alap szintaxis a következőképpen néz ki:

```
useEffect(() => {  
  // Side effect logic here  
});
```

- Alapértelmezés szerint a useEffect minden renderelés után lefut, de a működése szabályozható az úgynevezett függőségekkel (dependencies).

Mi az a useEffect?

A useEffect hook használata

1. Lefutás minden renderelés után

Ha nem adunk meg függőségeket, a useEffect minden renderelés után lefut (beleértve a frissítéseket is):

Ebben a példában a console.log minden alkalommal végrehajtódik, amikor a komponens újrenderelődik.

```
import React, { useEffect } from 'react';
import { Text, View } from 'react-native';

const App = () => {
  useEffect(() => {
    console.log('Component rendered');
  });

  return (
    <View>
      <Text>Hello, React Native!</Text>
    </View>
  );
};

export default App;
```

A useEffect hook használata

2. Lefutás csak egyszer

Ha a **componentDidMount** viselkedését szeretnénk elérni, adjunk meg egy üres függőségi tömböt (**[]**):

```
useEffect(() => {  
  console.log('This runs only once after the component mounts');  
}, []);
```

Ez a megoldás különösen hasznos kezdeti beállításoknál, például adatlekéréseknél vagy valamilyen szolgáltatás inicializálásánál.

A useEffect hook használata

3. Lefutás megadott függőségek esetén

A useEffect csak akkor is lefuthat, amikor bizonyos értékek megváltoznak. Ehhez ezeket az értékeket fel kell venni a függőségi tömbbe:

```
useEffect(() => {  
  console.log('Dependency changed!');  
}, [someDependency]);
```

Ez a megoldás különösen hasznos, ha prop- vagy state-változásokra szeretnénk reagálni.

A useEffect hook használata

4. Cleanup logika

Bizonyos side effectek – például időzítők vagy eseményfeliratkozások – esetén szükség van takarításra (cleanup). A useEffect lehetővé teszi, hogy egy cleanup függvényt adjunk vissza:

Ez biztosítja, hogy ne maradjanak fölöslegesen futó folyamatok, és elkerülhetők legyenek a memóriahibák vagy nem kívánt működések.

```
useEffect(() => {  
  const interval = setInterval(() => {  
    console.log('Interval running');  
  }, 1000);  
  
  return () => {  
    clearInterval(interval);  
    console.log('Cleanup function called');  
  };  
}, []);
```

Nagyon egyszerű useEffect példa

```
import React, { useState, useEffect } from 'react';
import { View, Text, Button } from 'react-native';

export default function App() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log('A count értéke megváltozott:', count);
  }, [count]);

  return (
    <View
      style={{
        flex: 1,
        alignItems: 'center',
        justifyContent: 'center',
      }}
    >
      <Text style={{ fontSize: 24 }}>Számláló: {count}</Text>

      <Button title="Növelés" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```



PROBLEMS LOGS

sdk_gphone64_x86_64: A count értéke megváltozott: 0
sdk_gphone64_x86_64: A count értéke megváltozott: 1
sdk_gphone64_x86_64: A count értéke megváltozott: 2
sdk_gphone64_x86_64: A count értéke megváltozott: 3
sdk_gphone64_x86_64: A count értéke megváltozott: 4
sdk_gphone64_x86_64: A count értéke megváltozott: 5
sdk_gphone64_x86_64: A count értéke megváltozott: 6
sdk_gphone64_x86_64: A count értéke megváltozott: 7
sdk_gphone64_x86_64: A count értéke megváltozott: 8
sdk_gphone64_x86_64: A count értéke megváltozott: 9
sdk_gphone64_x86_64: A count értéke megváltozott: 10
sdk_gphone64_x86_64: A count értéke megváltozott: 11
sdk_gphone64_x86_64: A count értéke megváltozott: 12

Számláló: 12

NÖVELÉS

Paraméterek átadása beágyazott (nested) screennek



```
navigation.navigate('More', {  
  screen: 'Settings',  
  params: { userId: 'jane' },  
})
```

- Ha egymásba ágyazott navigatorokat használsz, a paraméterek átadása ugyanúgy történik, mint amikor egy beágyazott screenre navigálsz.
- Beágyazott screen-ekről és beágyazott navigátorról a későbbiekben lesz szó részletesen

Fenntartott paraméternevek

- Bizonyos paraméterneveket a React Navigation belső működésre használ, különösen beágyazott navigatorok esetén:
 - **screen**
 - **params**
 - **initial**
 - **state**
- Ezeket a neveket érdemes elkerülni a saját kódodban. Ha mégis ilyen nevű paramétereket használsz, az könnyen váratlan működéshez vezethet.
- A szülő (parent) screenekben ezeknek a paramétereknek az olvasása kifejezetten nem ajánlott.

Mit érdemes a params-ba tenni?

- A params objektumnak csak a screen megjelenítéséhez feltétlenül szükséges, minimális információt kell tartalmaznia:
 - Azonosítók, amelyek meghatározzák, mit kell megjeleníteni (például userId, itemId)
 - A screenhez tartozó állapotinformációk (például rendezési sorrend, szűrők, oldalszám)
- A params-ra érdemes úgy gondolni, mint az URL query paraméterekre: azonosítókat és állapotot tartalmazzanak, ne teljes adatobjektumokat.
- A tényleges adatokat inkább globális state-ből vagy cache-ből célszerű betölteni.

Mit érdemes a params-ba tenni?

- Ezután a felhasználó adatait az ID alapján kérjük le egy globális cache-ből vagy egy API-ból.
- Az adatlekérést és a gyorsítótárazást olyan könyvtárak is megkönnyítik, mint például a React Query.

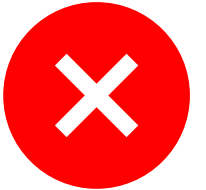
Jó példák params használatára:

- Azonosítók (ID-k): `navigation.navigate('Profile', { userId: 'jane' })`
- Rendezés / szűrés: `navigation.navigate('Feeds', { sortBy: 'latest' })`
- Lapozás (pagination): `navigation.navigate('Chat', { beforeTime: 1603897152675 })`
- Bemeneti adatok (input): `navigation.navigate('ComposeTweet', { title: 'Hello world!' })`

Mit érdemes a params-ba tenni?

- Például, ha van egy Profile screened, elsőre logikusnak tűnhet a teljes user objektum átadása params-ként:
- Ez nem ajánlott megoldás (anti-pattern), mert:
 - Az adatok duplikálódnak, ami könnyen elavult (stale) adatokhoz és hibákhoz vezethet
 - Minden screennek tudnia kellene, hogyan kérje le a felhasználói adatokat
 - Az URL-ek vagy deep linkek a teljes objektumot tartalmazzák, ami problémákat okozhat
- Ehelyett érdemes csak az azonosítót (ID-t) átadni:

```
// Don't do this  
navigation.navigate('Profile', {  
  user: {  
    id: 'jane',  
    firstName: 'Jane',  
    lastName: 'Dane',  
    age: 25,  
  },  
});
```



```
navigation.navigate('Profile', { userId: 'jane' });
```



A header bar konfigurálása

```
const MyStack = createNativeStackNavigator({
  screens: {
    Home: {
      screen: HomeScreen,
      options: {
        title: 'My home',
      },
    },
  },
});
```

A header címének beállítása

- Minden screen rendelkezik egy **options** tulajdonsággal (ez lehet objektum vagy olyan függvény, amely objektumot ad vissza), amely a navigator viselkedésének és megjelenésének konfigurálására szolgál.
- A header címének megadásához a **title** opciót használhatjuk.

Paraméterek használata a címben

- Ha a params értékeit szeretnénk megjeleníteni a header címében, az **options** tulajdonságot függvényként kell megadni, amely egy konfigurációs objektumot ad vissza.
- A React Navigation ezt a függvényt a **{ navigation, route }** objektummal hívja meg, így a paraméterek a **route.params** segítségével érhetők el.
- Az options függvény egy olyan objektumot kap paraméterként, amely a következő tulajdonságokat tartalmazza:
 - **navigation**: A screenhez tartozó navigation objektum
 - **route**: A screenhez tartozó route objektum
- A példában csak a route objektumra volt szükségünk, de bizonyos esetekben a navigation objektumot is hasznos lehet felhasználni.

```
const MyStack = createNativeStackNavigator({
  screens: {
    Home: {
      screen: HomeScreen,
      options: {
        title: 'My home',
      },
    },
    Profile: {
      screen: ProfileScreen,
      options: ({ route }) => ({
        title: route.params.name,
      }),
    },
  },
});
```

Opciók frissítése a **setOptions** segítségével

- A header beállításait a screen komponensen belül is módosíthatjuk a **navigation.setOptions** metódus használatával.

```
<Button  
  onPress={() =>  
    navigation.setOptions({ title: 'Updated!' })  
  }  
>  
  Update the title  
</Button>
```

A header stílusának módosítása

```
const MyStack = createNativeStackNavigator(  
  screens: {  
    Home: {  
      screen: HomeScreen,  
      options: {  
        title: 'My home',  
        headerStyle: {  
          backgroundColor: '#f4511e',  
        },  
        headerTintColor: '#fff',  
        headerTitleStyle: {  
          fontWeight: 'bold',  
        },  
      },  
    },  
  },  
});
```

A header megjelenésének testreszabásakor három fontos beállítást érdemes ismerni:

- **headerStyle:** Egy stílusobjektum, amely a headert körülvevő nézetre (view) vonatkozik. Ha itt például **backgroundColor** értéket adunk meg, az határozza meg a header háttérszínét.
- **headerTintColor:** Meghatározza a vissza gomb és a cím színét. A példában fehérre (#fff) állítjuk, így a back gomb és a header címe is fehér lesz.
- **headerTitleStyle:** A header címének szövegstílusát szabályozza. Itt állíthatunk be például **fontFamily**, **fontWeight** és egyéb szöveghez tartozó stílusokat.
- A megadott konfiguráció csak a Home screenre érvényes. Amikor a Details screenre navigálunk, a header visszaáll az alapértelmezett stílusra.

Az összes screen testreszabása

- Gyakran előfordul, hogy egy navigator összes screenjén ugyanazokat a beállításokat szeretnénk alkalmazni.
- Ahelyett, hogy minden screennél külön megismételnénk ezeket, használhatjuk a navigator **screenOptions** tulajdonságát.
- Ebben az esetben a navigator minden screenje ezeket a stílusokat fogja használni.
- Az egyes screenek azonban továbbra is felülírhatják a közös beállításokat a saját **options** megadásával.

```
const MyStack = createNativeStackNavigator({
  screenOptions: {
    headerStyle: {
      backgroundColor: '#f4511e',
    },
    headerTintColor: '#fff',
    headerTitleStyle: {
      fontWeight: 'bold',
    },
  },
  screens: {
    Home: {
      screen: HomeScreen,
    },
    Details: {
      screen: DetailsScreen,
    },
  },
});
```

A cím lecserélése egy egyedi komponensre

- Előfordulhat, hogy a header címének egyszerű szöveges módosítása nem elegendő.
- Például lehet, hogy a cím helyén egy képet szeretnénk megjeleníteni, vagy a címet gombként szeretnénk használni.
- Ilyen esetekben teljesen lecserélhetjük a cím megjelenítéséért felelős komponenst, és saját komponenst adhatunk meg.

A **headerTitle** kifejezetten a headerhez tartozik, míg a **title** beállítást más navigációs elemek is használhatják (például tab bar, drawer, vagy weben az oldal címe). Alapértelmezés szerint a **headerTitle** a **title** értékét jeleníti meg egy Text komponensben.

```
function LogoTitle() {
  return (
    <Image
      style={{ width: 50, height: 50 }}
      source={require('@expo/snack-static/react-native-logo.png')}
    />
  );
}

const MyStack = createNativeStackNavigator({
  screens: {
    Home: {
      screen: HomeScreen,
      options: {
        headerTitle: (props) => <LogoTitle {...props} />,
      },
    },
  },
});
```

További beállítások

- A headerrel kapcsolatos összes lehetőség a `createNativeStackNavigator` dokumentációjában található.
- <https://reactnavigation.org/docs/native-stack-navigator/#options>

